

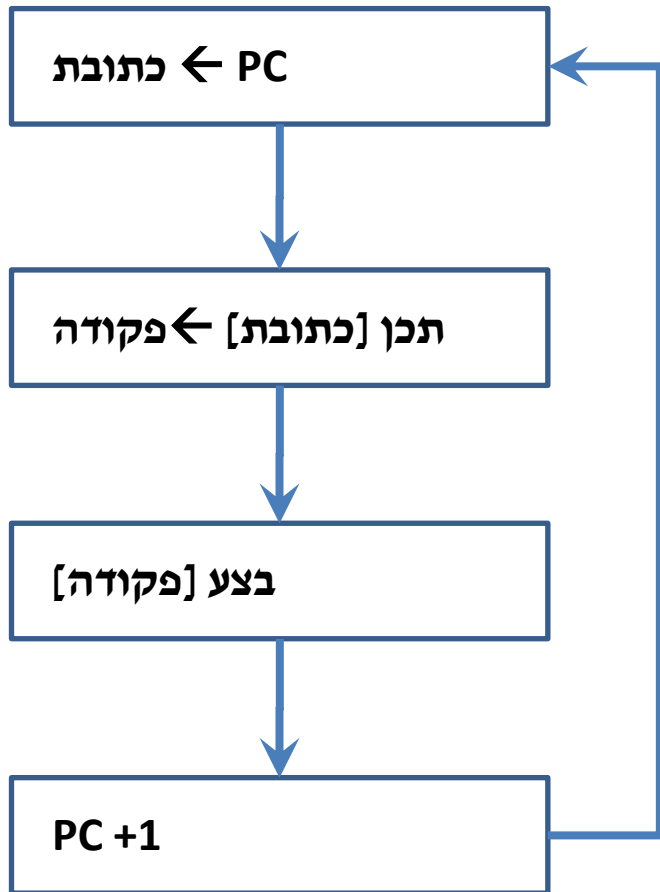


חוטאים Threads

יחידה 9

מעבד מרכזי (CPU)

■ מה הוא עושה כל היום?



מה בתכנית :

- קצת הסטוריה – איך נולד החוט
- קצת ג'אווה – איך טווים חוט בג'אווה
- קצת חכמת חיים – למה צריכים חוטים?
- קצת מציאות עגומה – צער גידול חוטים

מהו חוט Thread?

■ תהליך בו מחשב מבצע סדרת פקודות של תכנית
כמו הביצוע של רוב התכניות שכתבתם עד כה ■



החוט – מאז ועד היום

■ בראשית היה המחשב

■ הזינו תכנית (בינארית) לזכרון ...

● אולי מכרטיס מנוקב, אולי מסרט מגנטי

■ ... ולחצו על כפתור  ...

■ ... והמחשב רץ עד שבוצעה פקודת `halt`

■ הפקודות בוצעו בזו אחר זו לפי הסדר של ביצוע התכנית

● לכן קראו לזה תכנית

■ כל זה מהווה חוט יחיד

החוט – מאז ועד היום

■ בראשית היה המחשב

■ ואז נבראה מערכת ההפעלה

■ היא היתה התכנית הראשונה שהופעלה

■ ... והיתה גם האחרונה להתבצע

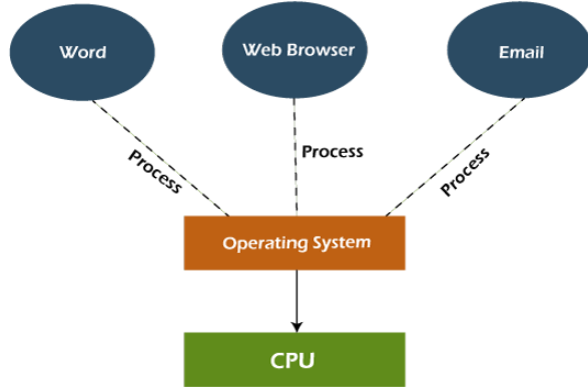
■ בין לבין, תכניות של משתמשים בוצעו כתת-תכניות של מערכת ההפעלה

◆ משהסתיימה תכנית אחת, מערכת ההפעלה בחרה אחרת

■ עדיין, הביצוע כלל **חוט יחיד** מתחילת יום העבודה ועד סופו

◆ בסוף היום כיבו את המחשב

החוט – מאז ועד היום



■ בראשית היה המחשב

■ ואז נבראה מערכת ההפעלה

■ ואז המציאו את ה-multitasking

כל תכנית משתמש נמצאת בתוך תהליך – process

כל תהליך מוגדרים משאבים משלו :

◆ שטח זכרון

◆ חלונות על המסך

◆ קבצים, חיבורי רשת, ציוד פריפרי בשימוש

כל המערכת תומכת בכמה תהליכים פעילים בו זמנית

◆ משאבים (CPU, למשל) ניתנים לכל תהליך לזמן קצוב לפי תור (TDM)

■ עכשיו, כל תהליך הוא חוט

Context Switch

■ איך אפשר לעבור מסדרת ביצוע אחת לאחרת ולשוב חזרה לראשונה מבלי להתבלבל?

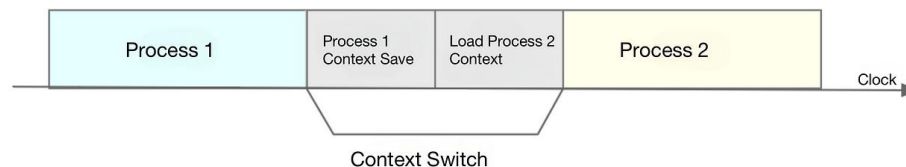
■ מנגנון חמרה

■ מאפשר לטעון את כל הרגיסטרים בערכים המצויים בזכרון במקום ידוע

■ מצב הרגיסטרים של התהליך נשמר בזיכרון קבוע השייך לו

■ מאפשר לחזור לביצוע אותו הפסקנו בדיוק מאותו המקום

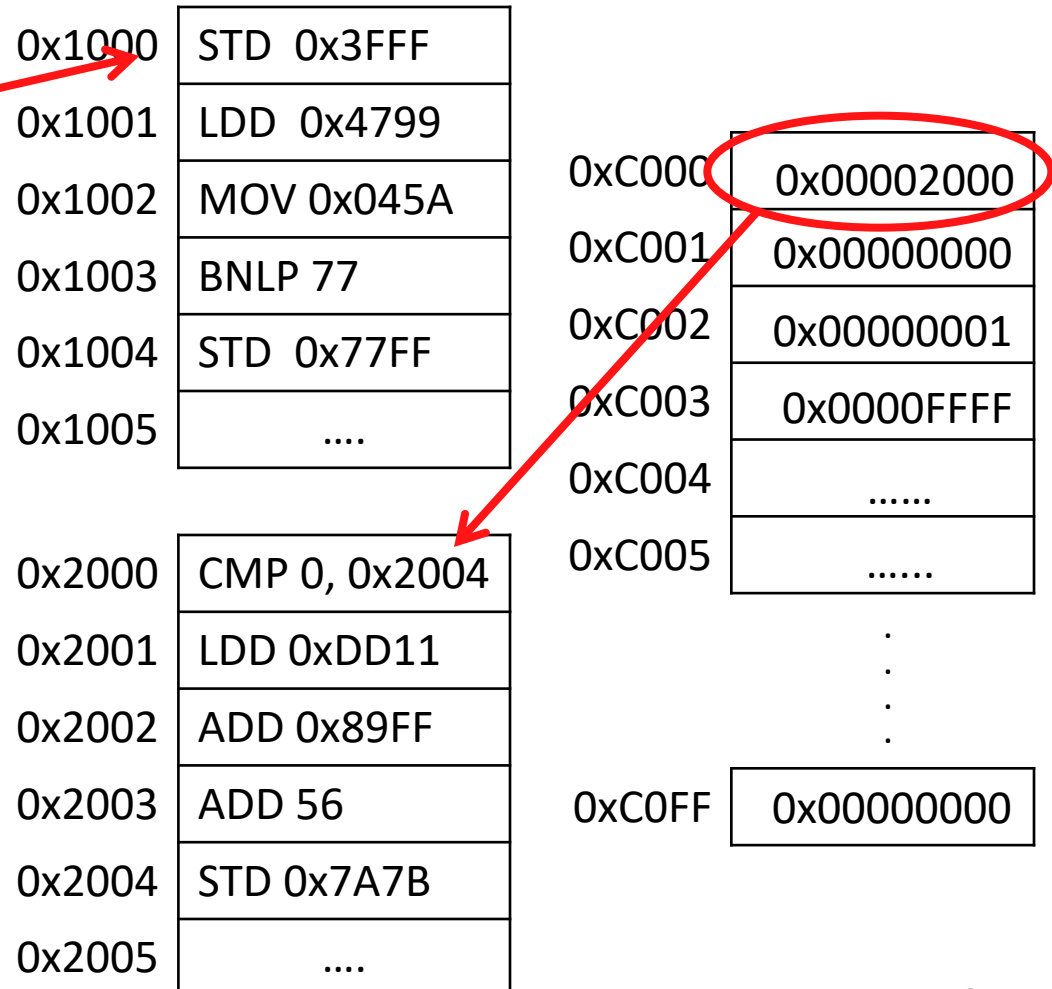
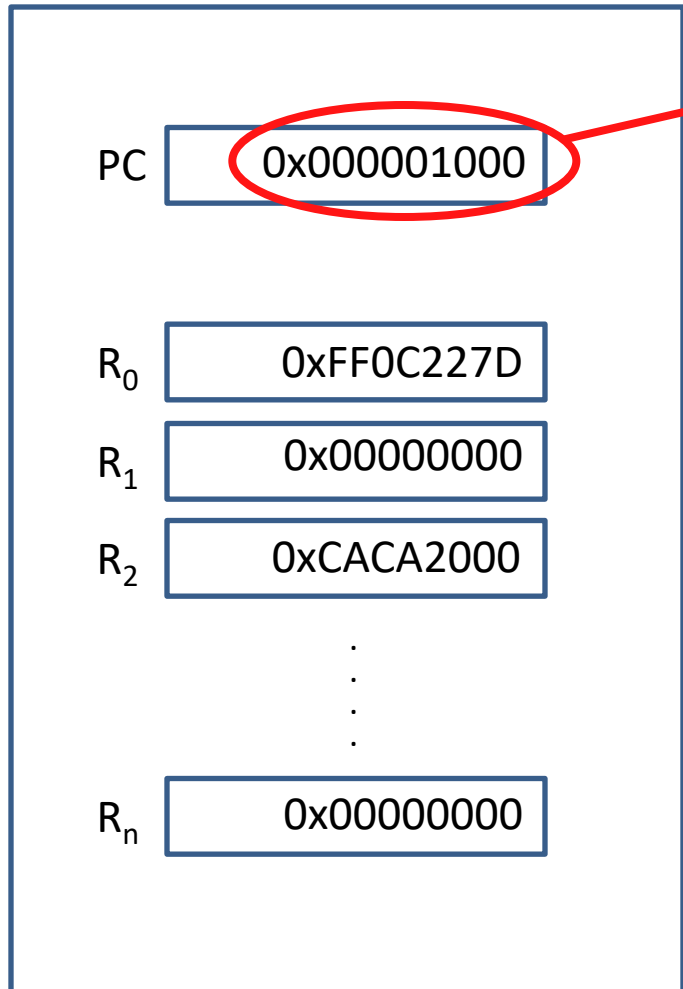
■ בניגוד למנגנון הקודם, מופעל ע"י תכנה (אינו מופעל ע"י כפתור).



Context Switch

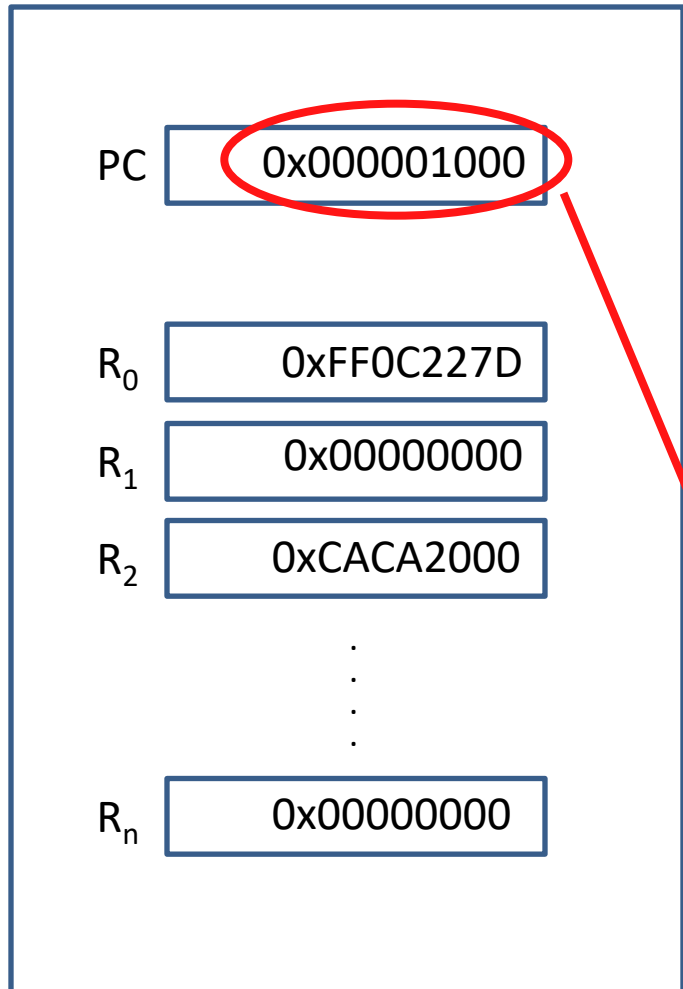
CPU

RAM



Context Switch

CPU



RAM

0x1000	STD 0x3FFF
0x1001	LDD 0x4799
0x1002	MOV 0x045A
0x1003	BNLP 77
0x1004	STD 0x77FF
0x1005	
0x2000	CMP 0, 0x2004
0x2001	LDD 0xDD11
0x2002	ADD 0x89FF
0x2003	ADD 56
0x2004	STD 0x7A7B
0x2005	

0xC000	0x00002000
0xC001	0x00000000
0xC002	0x00000001
0xC003	0x0000FFFF
0xC004	
0xC005	
...	...
0xC0FF	0x00000000

החוט – מאז ועד היום

- בראשית היה המחשב
- ואז נבראה מערכת ההפעלה
- ואז המציאו את ה-multitasking
- ולבסוף המציאו את ה-multithreading
- לחוט יש רק מה שנחוץ על מנת לרוץ בנפרד:
 - ◆ מחסנית (לעקוב אחרי הפעלות של סוברוטיות)
 - ◆ מצב הרגיסטרים
- מערכת ההפעלה (עדיין) נותנת יחידת זמן ביצוע לכל חוט
- בתהליך אחד יתכנו כמה חוטים

למה כל זה טוב?

■ אלגוריתמים שטבעי לבטא באמצעות רצפי פקודות נפרדים

■ סימולציות

■ מערכות מגיבות (שומרות על קשר התגובה גם כשהן עסוקות)

◆ שרתים (במערכת שרת-לקוח)

◆ GUI (ממשק משתמש)

■ ניצול יכולות עבוד מקבילי

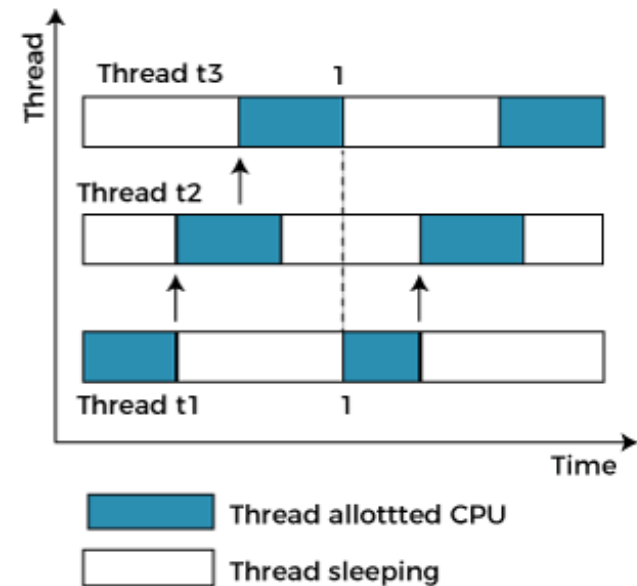
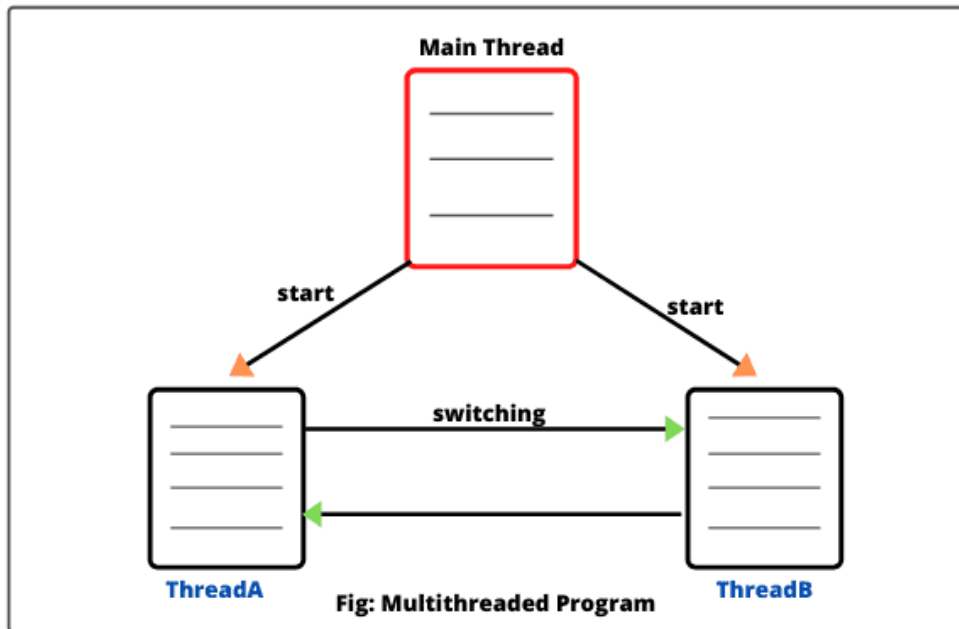
hyperthreading ■

multicore ■

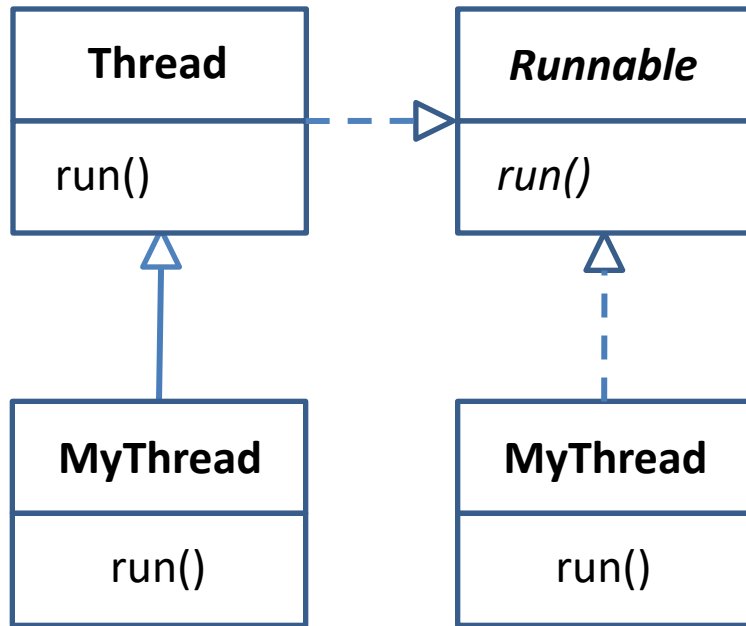
multiprocessor ■

cluster ■

הדגמות של ריצת חוטים וcontext switch



איך זה נעשה ב-Java?



■ שתי דרכים להגדיר חוט

● הרחבה של `Thread`

● מימוש של `Runnable`

■ מתודה `run()`

● מכילה את מה שהחוט אמור לבצע

◆ מקבילה ל-`main( String[] args )`

● ל-`Thread` יש מימוש ריק: חובה לעשות `Override`

■ מתודה `start()`

● מתחילה חוט חדש קוראת ל-`run()` שהוגדר לעיל להתבצע עליו

● הביצוע מסתיים כאשר `run()` חוזרת

ירושה מ-Thread

```
01 public class Counter extends Thread
02 {
03     private int count = 0;
04     private int inc = 0;
05     private int delay = 0;
06
07     public Counter( int init, int inc, int delay )
08     {
09         this.count = init;
10         this.inc = inc;
11         this.delay = delay;
12     }
13
14     public static void main( String[] args )
15     {
16         Counter c1 = new Counter( 0, 1, 33 );
17         Counter c2 = new Counter( 0, -1, 100 );
18
19         c1.start();
20         c2.start();
21     }
22
```

```
23     public void run()
24     {
25         try
26         {
27             int cycles = 10000 / delay;
28             String space = ( delay < 60 ) ? " " : " ";
29
30             while( cycles-- > 0 )
31             {
32                 System.out.println( space + count );
33                 count += inc;
34                 Thread.sleep( delay );
35             }
36         }
37         catch( InterruptedException ie )
38         { /* do nothing */ }
39     }
40
41 } /* class Counter */
```

מימוש Runnable

מה אם המחלקה יורשת
ממשהו אחר?

```
01 public class Counter implements Runnable
02 {
03     private int count = 0;
04     private int inc = 0;
05     private int delay = 0;
06
07     public Counter( int init, int inc, int delay )
08     {
09         this.count = init;
10         this.inc = inc;
11         this.delay = delay;
12     }
13
14     public static void main( String[] args )
15     {
16         Counter c1 = new Counter( 0, 1, 33 );
17         Thread t1 = new Thread( c1 );
18         Counter c2 = new Counter( 0, -1, 100 );
19         Thread t2 = new Thread( c2 );
20
21         t1.start();
22         t2.start();
23     }
24
```

```
25     public void run()
26     {
27         try
28         {
29             int cycles = 10000 / delay;
30             String space = ( delay < 60 ) ? " " : " ";
31
32             while( cycles-- > 0 )
33             {
34                 System.out.println( space + count );
35                 count += inc;
36                 Thread.sleep( delay );
37             }
38         }
39         catch( InterruptedException ie )
40         { /* do nothing */ }
41     }
42
43 } /* class Counter */
```


מימוש מחלקה אנונימית

```
01 public class Counter {
02     public static void main(String[] args) {
03         Thread t1 = new Thread(() -> runCounter(0, 02 1, 33));
04         Thread t2 = new Thread(() -> runCounter(0, -1, 100));
05
06         t1.start();
07         t2.start();
08     }
```

מה אם לא נרצה
להגדיר מחלקה?

```
25     private static void runCounter(int init, int inc, int delay) {
26     {
27         try
28         {
29             int cycles = 10000 / delay;
30             String space = ( delay < 60 ) ? " " : " ";
31             int count = init;
32             while( cycles-- > 0 )
33             {
34                 System.out.println( space + count );
35                 count += inc;
36                 Thread.sleep( delay );
37             }
38         }
39         catch( InterruptedException ie )
40         { /* do nothing */ }
41     }
42
43 } /* class Counter */
```

שימוש ב-Executor

```
01 public class Counter implements Runnable
02 {
03     private int count = 0;
04     private int inc = 0;
05     private int delay = 0;
06
07     public Counter( int init, int inc, int delay )
08     {
09         this.count = init;
10         this.inc = inc;
11         this.delay = delay;
12     }
13
14     public static void main( String[] args )
15     {
16         ExecutorService es =
17             Executors.newCachedThreadPool();
18         es.execute( new Counter( 0, 1, 33 ) );
19         es.execute( new Counter( 0, -1, 100 ) );
20         es.shutdown();
21     }
22
```

```
23     public void run()
24     {
25         try
26         {
27             int cycles = 10000 / delay;
28             String space = ( delay < 60 ) ? " " : " ";
29
30             while( cycles-- > 0 )
31             {
32                 System.out.println( space + count );
33                 count += inc;
34                 Thread.sleep( delay );
35             }
36         }
37         catch( InterruptedException ie )
38         { /* do nothing */ }
39     }
40
41 } /* class Counter */
```

למחלקה Executors יש כמה מתודות סטטיות ליצירת

אובייקט מסוג *ExecutorService* לפי הסוג המבוקש, למשל:

- *newFixedThreadPool* המגדיר כמה חוטים יוכלו לרוץ במקביל

- *newCachedThreadPool* המאפשר שימוש בלתי מוגבל של

חוטים ויודע לעשות שימוש חוזר בזכרון של חוטים שכבר

סיימו עבודתם (במקום יצירת אובייקט חדש בכל פעם)

תמונת מראה עצמים

שליטה על חוטים

- כללית, אין למתכנת שליטה על ביצוע של חוטים
- מערכת ההפעלה מחליטה איזה חוטים לבצע ולכמה זמן
- יש שליטה מסויימת על מצב החוט (thread **state**)

מצב של חוט

■ חוט נמצא תמיד באחד מארבעה מצבים אפשריים:

new – חדש

◆ משנוצר ועד שנקרא start()

runnable – בר-ריצה (שלא להתבלבל עם המנשק Runnable)

◆ פעיל – מערכת ההפעלה יכולה להפעיל את החוט לפי שיקוליה

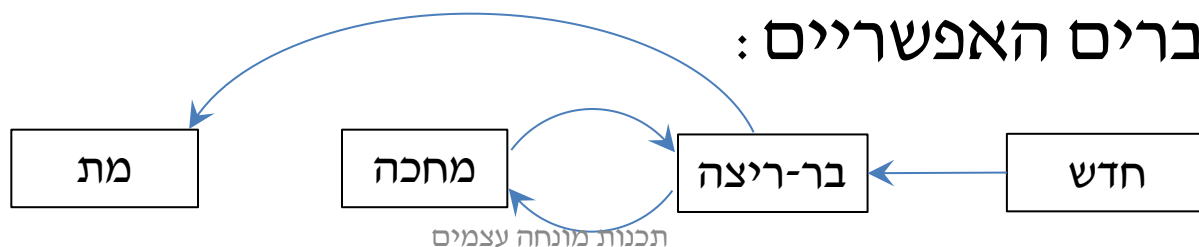
waiting, blocked – מחכה

◆ מחכה – לא יהפוך פעיל עד שהארוע לו החוט מחכה אכן יקרה

dead – מת

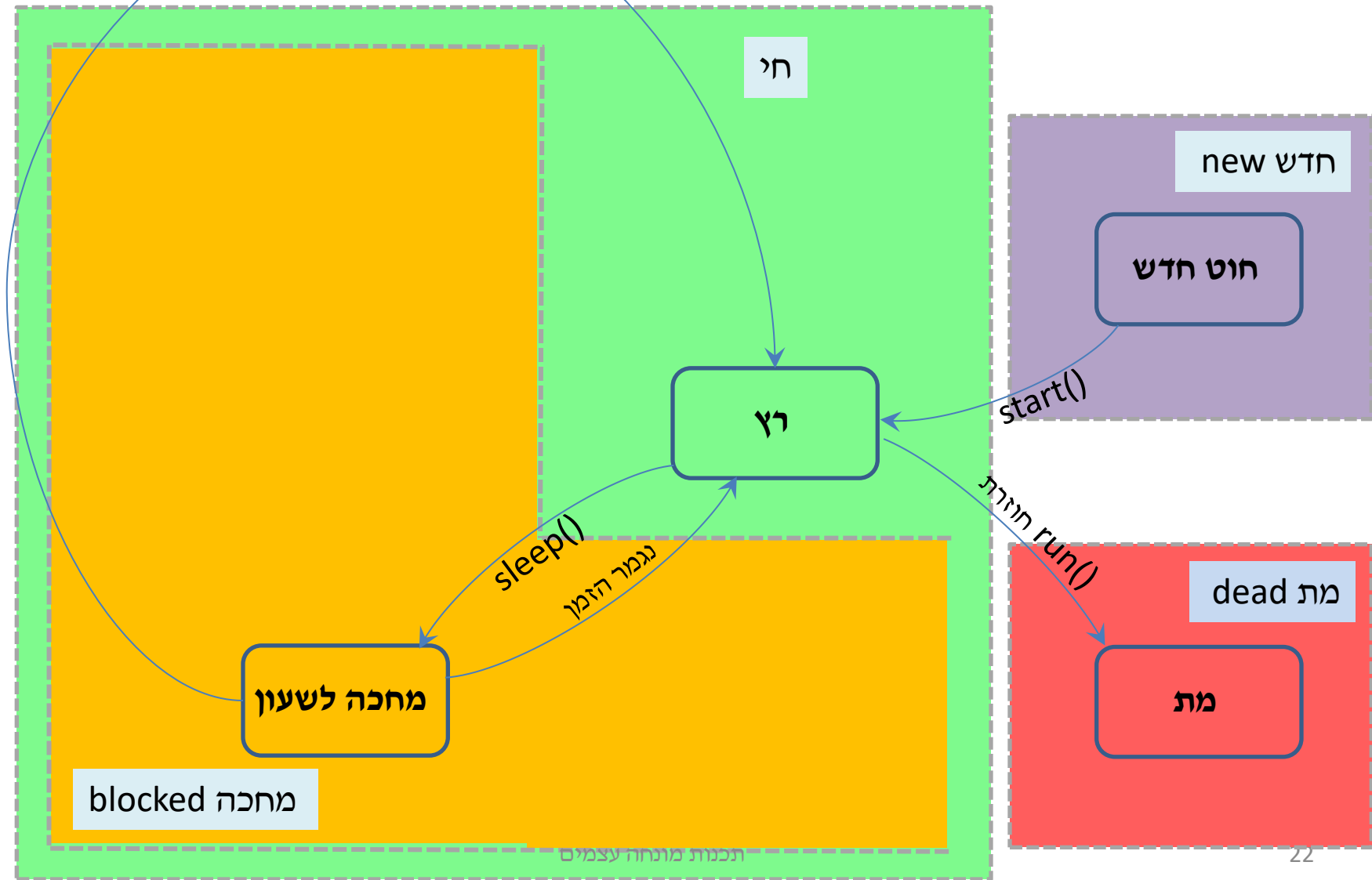
◆ עליו השלום

■ המעברים האפשריים:



דיאגרמת המצבים

interrupt()



Interrupt – אמצעי לעצירת חוט חי

- לכל חוט יש משתנה boolean בו נשמר מצב ה-interrupt
- המתודה interrupt() של החוט הופכת משתנה זה ל-true
 - כל אחד יכול לקרוא למתודה זו
- חוט יכול לברר את מצב המשתנה שלו עם interrupted()
 - קריאה למתודה זו הופכת את המשתנה ל-false
- אפשר לברר את מצבו של חוט אחר עם isInterrupted()
 - קריאה זו אינה משנה את מצב המשתנה
- מתודות הדורשות זמן ארוך לרוב זורקות חריגה
 - sleep(), join(), wait()
 - אחרת, כדאי להשתמש ב-interrupted()

join()

■ מחכה עד שיסתיים החוט
שה-join() שלו נקרא

```
01 public class JoinTest extends Thread
```

```
02 {
```

חוט צאצא

```
03     public void run()
```

```
04     {
```

```
05         for( int i = 0; i < 5; i++ )
```

```
06         {
```

```
07             System.out.println( "Child: working..." );
```

```
08             try
```

```
09             {
```

```
10                 Thread.sleep(1000);
```

```
11             }
```

```
12             catch ( InterruptedException ie ) {}
```

```
13         }
```

```
14     }
```

```
15
```

חוט הורה

(initial thread)

```
16     public static void main( String[] args )
```

```
17     {
```

```
18         JoinTest j = new JoinTest();
```

```
19         j.start();
```

```
20         try
```

```
21         {
```

```
22             System.out.println( "Initial thread: about to wait for child thread");
```

```
23             j.join();
```

```
24         }
```

```
25         catch ( InterruptedException ie ) {}
```

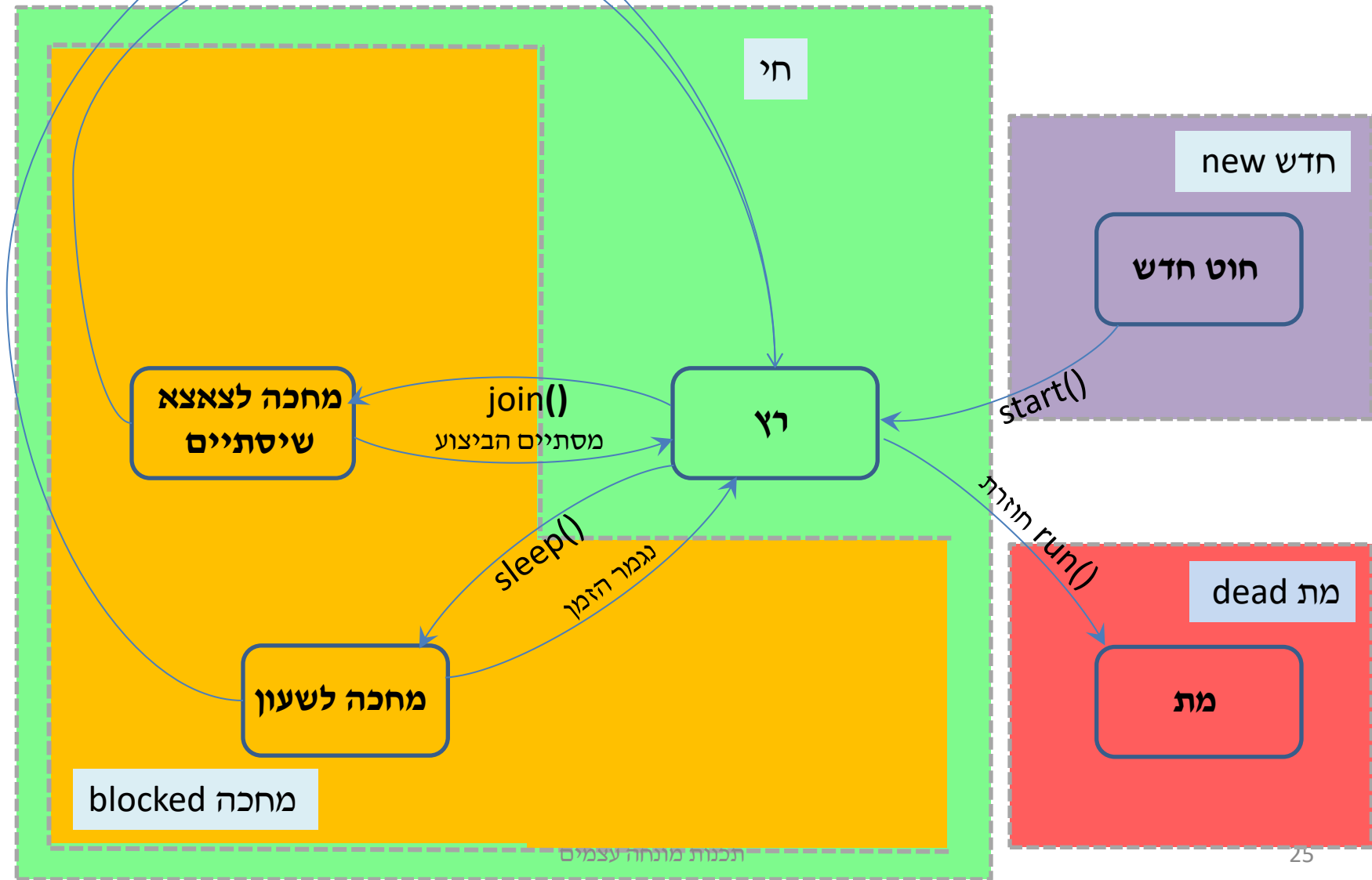
```
26         System.out.println( "Initial thread: joined with child thread" );
```

```
27     }
```

```
28 }
```

דיאגרמת המצבים

interrupt()



שאלון

Producer - Consumer

■ שני תהליכים אסינכרוניים :

■ יצרן (producer) – מפיק דבר-מה

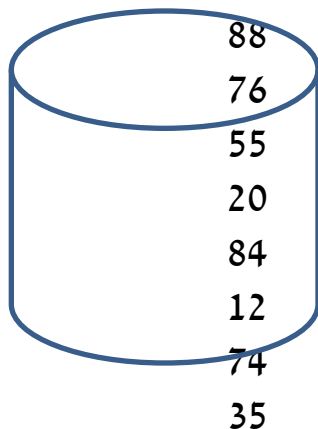
■ צרכן (consumer) – מנצל את אותו דבר-מה

■ מיכל (buffer) – מחזיק את אותו דבר-מה מהרגע שיוצר ועד שנצרך

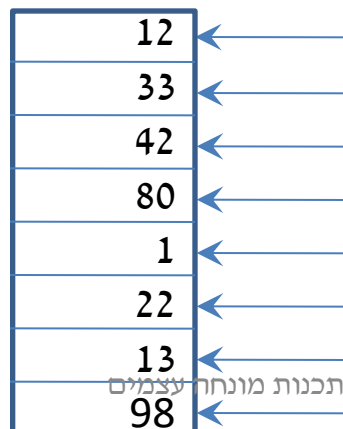
■ דוגמאות :

■ buffered I/O

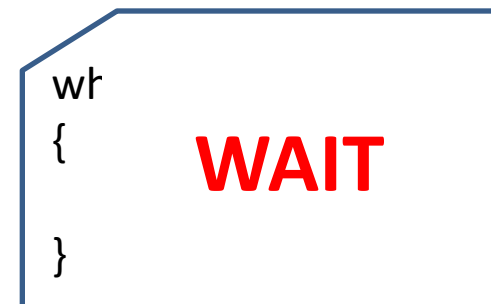
דיסק Producer



חיץ Buffer



אפליקציה Consumer



Producer - Consumer

■ שני תהליכים אסינכרוניים :

יצרן (producer) – מפיק דבר-מה

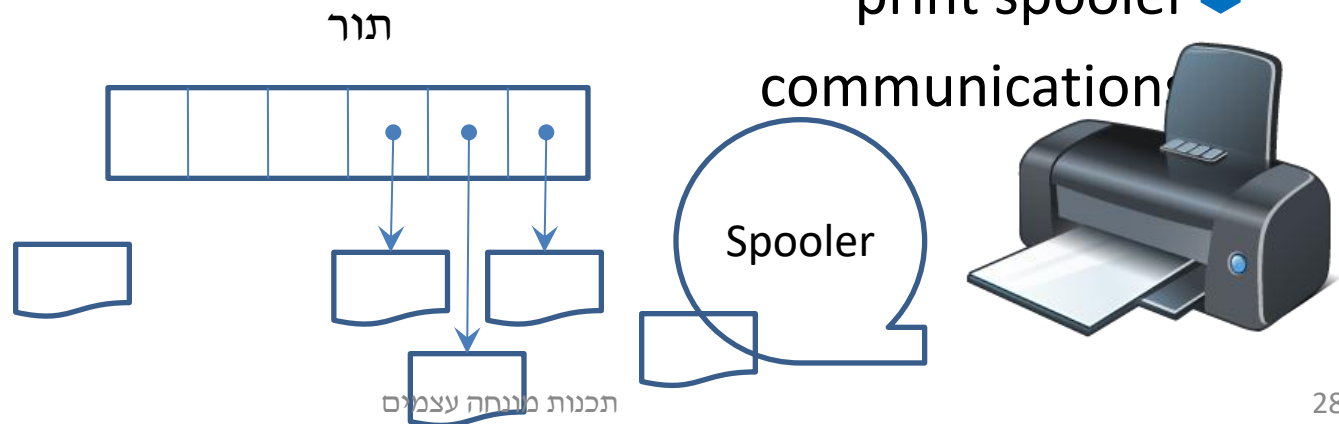
צרכן (consumer) – מנצל את אותו דבר-מה

מיכל (buffer) – מחזיק את אותו דבר-מה מהרגע שיוצר ועד שנצרך

■ דוגמאות :

buffered I/O

print spooler



דוגמא

■ יצרן

● כותב מספר לחיץ

● מדפיס את סכום המספרים שנכתבו עד כה לחיץ

■ צרכן

● קורא מספר מהחיץ

● מדפיס את סכום המספרים שנקראו עד כה

■ חיץ

● ננסה מימושים שונים של החיץ ונבדוק את התוצאות כל פעם

● נתפלא (אולי) לגלות שיש הבדלים בין מימושים שונים

דוגמה (1) : אופן הפעולה

Producer

מבצע מ-1 עד 20 :

❖ כותב לחייץ מספר

❖ מדפיס את סכום המספרים

שנכתבו עד כה

Buffer

אזור זכרון פסיבי



Consumer

מבצע מ-1 עד 20 :

❖ קורא מהחייץ מספר

❖ מדפיס את סכום המספרים

שנקראו עד כה

סכום המספרים
שנכתבו

6

3

סכום המספרים
שנקראו

6

דוגמה (2) : ממשק של החיץ (Buffer)

```
public interface Buffer
{
    /**
     * place int value into Buffer
     */
    public void set( int value ) throws InterruptedException;

    /**
     * obtain int value from Buffer
     */
    public int get() throws InterruptedException;
}
```

דוגמה (3) : יצרן (Producer)

```
public class Producer
    implements Runnable
{
    private Random generator =
        new Random();
    private Buffer sharedLocation;

    public Producer( Buffer shared ) {
        sharedLocation = shared;
    }

    public void run() {
        int sum = 0;
        for ( int count = 1; count <= 20; count++ )
        {
            try {
                Thread.sleep( generator.nextInt( 2000 ) );
                sharedLocation.set( count );
                sum += count;
                System.out.printf( "\t%2d\n", sum );
            }
            catch ( InterruptedException exception ) {
                exception.printStackTrace();
            }
        }
        System.out.println( "Producer done producing" );
        System.out.println( "Terminating Producer" );
    }
} /* end class */
```

דוגמה (4) : צרכן (Consumer)

```
public class Consumer
    implements Runnable
{
    private Random generator =
        new Random();
    private Buffer sharedLocation;

    public Consumer( Buffer shared ) {
        sharedLocation = shared;
    }

    public void run() {
        int sum = 0;
        for ( int count = 1; count <= 20; count++ )
        {
            try {
                Thread.sleep( generator.nextInt( 3000 ) );

                sum += sharedLocation.get();
                System.out.printf( "\t\t\t%2d\n", sum );
            }
            catch ( InterruptedException exception ) {
                exception.printStackTrace();
            }
        }
        System.out.println( "Total numbers read: " + sum );
        System.out.println( "Terminating Consumer" );
    }
}

/* end class */
```


דוגמה (5) : מימוש של החיץ – ללא תאום

```
public class UnsynchronizedBuffer implements Buffer
{
    private int buffer = -1;

    public void set( int value ) throws InterruptedException
    {
        System.out.printf( "Producer writes\t%2d", value );
        buffer = value;
    }

    public int get() throws InterruptedException
    {
        System.out.printf( "Consumer reads\t%2d", buffer );
        return buffer;
    }
}
```

דוגמה (5) main:

```
public class UnsynchronizedBufferTest
{
    public static void main( String[] args )
    {
        ExecutorService application = Executors.newCachedThreadPool();
        Buffer sharedLocation = new UnsynchronizedBuffer();

        System.out.println( "Action\t\tValue\tSum of Produced\tSum of Consumed" );
        System.out.println( "-----\t\t\t-----\t\t\t-----\n" );

        application.execute( new Producer( sharedLocation ) );
        application.execute( new Consumer( sharedLocation ) );

        application.shutdown();
    }
}
```

מהו הקוץ שבאליה?

■ מסקנה : בגישה לזכרון משותף יש סכנה למצבי מרוץ

■ שני חוטים מעדכנים את אותו מבנה הנתונים

■ עדכון לא-אטומי מסתכן במצבים בלתי קונסיסטנטיים

■ המחשה

■ כוכבים בשמיים

◆ אנו רואים כוכבים היום שכבו לפני שנים רבות

■ תפוח ביד

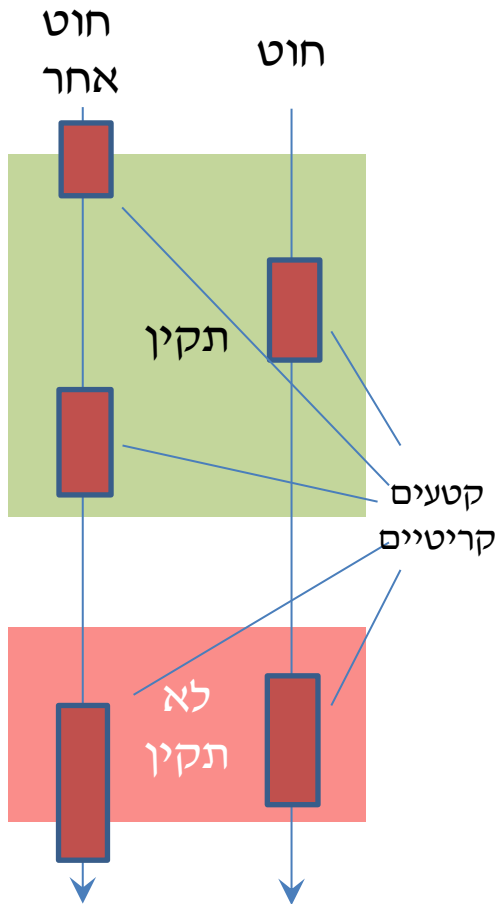
◆ אנו רואים את התפוח ביד, ומניחים שהוא שם

למרות שאפשר שהוא כבר לא

◆ זמן תגובה של מחשב הוא מסדר גודל של זמן מהלך הקרן

מהתפוח לעיננו

מהו הקוץ שבאליה?



■ קטע קוד המעדכן מבנה משותף קרוי
קטע קריטי (critical region)

■ הבעיה:

● על קטע קריטי להתבצע בשלמותו

● מבלי לאפשר לחוטים אחרים להתבצע במקביל

■ פתרון: סינכרוניזציה

סינכרוניזציה

■ לכל עצם קשור מנעול

■ רק חוט יחיד, לכל היותר, יכול לקבל חזקה על המנעול

■ כל בלוק ניתן לקשור למנעול

■ על מנת להכנס אליו, יש לקבל תחילה חזקה על המנעול

```
.....  
synchronized( instance )  
{  
    critical region  
}
```

=

```
.....  
{  
    obtain instance's lock  
    if unavailable, wait until it is available  
    do critical region  
    release instance's lock  
}
```

סינכרוניזציה

■ לכל עצם קשור מנעול

■ רק חוט יחיד, לכל היותר, יכול לקבל חזקה על המנעול

■ כל בלוק ניתן לקשור למנעול

■ על מנת להכנס אליו, יש לקבל תחילה חזקה על המנעול

■ אפשר לקשור גם מתודה שלמה

■ המנעול אז הוא זה הקשור ל-**this**

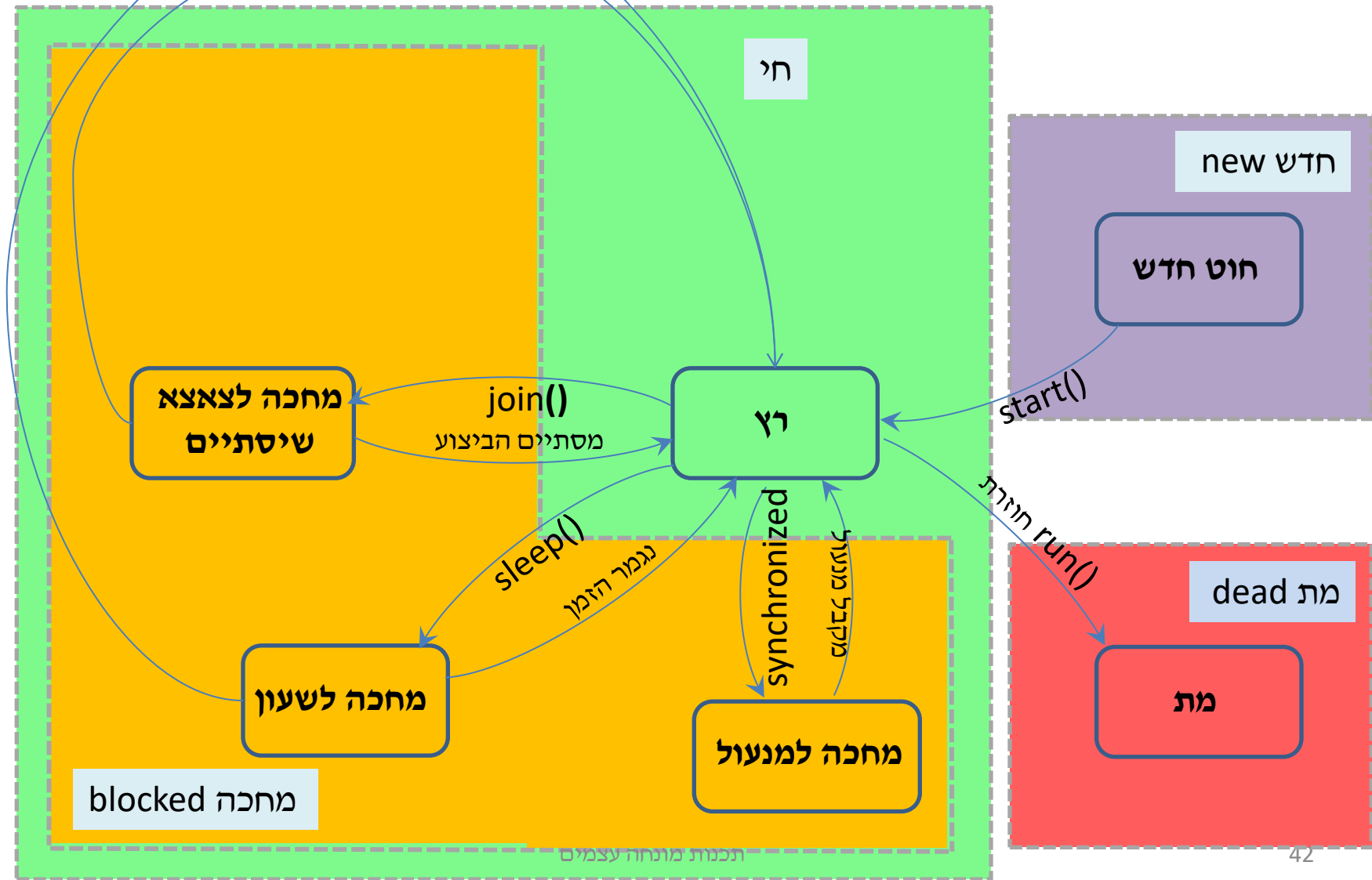
```
synchronized public void method()  
{  
    // critical region  
}
```



```
public void method()  
{  
    synchronized (this){  
        // critical region  
    }  
}
```

דיאגרמת המצבים

interrupt()



תותח בועות

■ התותח יורה 10000 בועות סבון בפרקי זמן אקראיים

■ כל בועה :

כל בועה מתבצעת בחוט נפרד { מוסיפה 1 למונה (סטטי)
מתקיימת זמן אקראי
מחסרת 1 מהמונה

■ מה מציין ערך המונה?

■ מה תצפו שיהיה ערכו בסוף?




```
public class Bubble extends Thread {
```

```
    public static int live = 0;
```

```
    private BubbleGun bg;
```

```
    private int exist;
```

```
    public Bubble( BubbleGun bg, int exist )
```

```
    {  
        this.bg = bg;
```

```
        this.exist = exist;
```

```
        incLive();
```

```
        start();  
    }
```

```
    private void incLive() {
```

```
        live++;  
    }
```

```
    private void decLive() {
```

```
        live--;  
    }
```

```
    public void run() {
```

```
        try {
```

```
            Thread.sleep(exist );  
        }
```

```
        catch( InterruptedException e ) { }
```

```
        decLive();  
    }
```

```
}
```



BubbleGun

```
public class BubbleGun extends Thread  
{
```

```
    public void run( ) {
```

```
        Set<Thread> bubbleThreads =
```

```
            new HashSet<Thread>();
```

```
        for( int i = 0; i < 10000; i++ ) {
```

```
            Thread b = new Bubble(this,normal(20,5));
```

```
            bubbleThreads.add(b);
```

```
            try { Thread.sleep( normal( 4, 2 ) ); }
```

```
            catch( InterruptedException e ) { }
```

```
        }
```

```
        for (Thread t : bubbleThreads) {
```

```
            try { if (t.isAlive()) t.join(); }
```

```
            catch (InterruptedException e) { }
```

```
        }
```

```
        System.out.println(Bubble.live);  
    }
```

```
    public static void main( String[] args ) {
```

```
        BubbleGun bg = new BubbleGun ();
```

```
        bg.start();  
    }
```

```
}
```

normal(a, b) - מחזיר מספר שלם רנדומלי

כאשר ההתפלגות היא נורמלית עם ממוצע a, ושונות b.

```
public class Bubble extends Thread {
```

```
    public static int live = 0;
```

```
    private BubbleGun bg;
```

```
    private int    life;
```

```
    public Bubble( BubbleGun bg, int life ) {
```

```
        this.bg = bg;
```

```
        this.life = life;
```

```
        incLive();
```

```
        start();
```

```
    }
```

```
    private synchronized void incLive() {
```

```
        live++;
```

```
    }
```

```
    private synchronized void decLive() {
```

```
        live--;
```

```
    }
```

```
    public void run() {
```

```
        try {
```

```
            sleep( life );
```

```
        }
```

```
        catch( InterruptedException e ) { }
```

```
        decLive();
```

```
    }
```

```
}
```



BubbleGun

```
public class BubbleGun extends Thread
```

```
{
```

```
    public void run( ) {
```

```
        Set<Thread> bubbleThreads =
```

```
            new HashSet<Thread>();
```

```
        for( int i = 0; i < 10000; i++ ) {
```

```
            Thread b = new Bubble(this,normal(20,5));
```

```
            bubbleThreads.add(b);
```

```
            try { sleep( normal( 4, 2 ) ); }
```

```
            catch( InterruptedException e ) { }
```

```
        }
```

```
        for (Thread t : bubbleThreads) {
```

```
            try { if (t.isAlive()) t.join(); }
```

```
            catch (InterruptedException e) { }
```

```
        }
```

```
        System.out.println(Bubble.live);
```

```
    }
```

```
    public static void main( String[] args ) {
```

```
        BubbleGun bg = new BubbleGun ();
```

```
        bg.start();
```

```
    }
```

```
}
```

```
public class Bubble extends Thread {
```

```
    public static int live = 0;
```

```
    private BubbleGun bg;
```

```
    private int    life;
```

```
    public Bubble( BubbleGun bg, int life ) {
```

```
        this.bg = bg;
```

```
        this.life = life;
```

```
        incLive();
```

```
        start();
```

```
    }
```

```
    private void incLive() {
```

```
        synchronized( bg ) { live++ ;};
```

```
    }
```

```
    private void decLive() {
```

```
        synchronized( bg ) { live-- ;};
```

```
    }
```

```
    public void run() {
```

```
        try {
```

```
            Thread.sleep( life );
```

```
        }
```

```
        catch( InterruptedException e ) { }
```

```
        decLive();
```

```
    }
```

```
}
```



BubbleGun

```
public class BubbleGun extends Thread  
{
```

```
    public void run( ) {
```

```
        Set<Thread> bubbleThreads =
```

```
            new HashSet<Thread>();
```

```
        for( int i = 0; i < 10000; i++ ) {
```

```
            Thread b = new Bubble(this,normal(20,5));
```

```
            bubbleThreads.add(b);
```

```
            try { Thread.sleep( normal( 4, 2 ) ); }
```

```
            catch( InterruptedException e ) { }
```

```
        }
```

```
        for (Thread t : bubbleThreads) {
```

```
            try { if (t.isAlive()) t.join(); }
```

```
            catch (InterruptedException e) { }
```

```
        }
```

```
        System.out.println(Bubble.live);
```

```
    }
```

```
    public static void main( String[] args ) {
```

```
        BubbleGun bg = new BubbleGun ();
```

```
        bg.start();
```

```
    }
```

```
}
```

למה טוב synchronize?

- מבטיח שרק חוט יחיד נוגע במשתנים משותפים
- אם יש כמה מהם שחשוב שישתנו ביחד, אפשר להבטיח זאת
- אבל איך להבטיח שיתוף פעולה בין כמה חוטים?

למה טוב synchronize?

- מבטיח שרק חוט יחיד נוגע במשתנים משותפים
- אם יש כמה מהם שחשוב שישתנו ביחד, אפשר להבטיח זאת
- מבטיח שרק חוט אחד יבצע עדכון באותו הזמן
- ◆ חוטים אחרים שמבקשים לעדכן יאלצו לחכות
- אבל איך להבטיח **סדר** ביצוע בין חוטים?
- איך להבטיח שהיצרן יפעל **לפני** הצרכן?

הבטחת סדר ביצוע

■ מתודות wait() ו-notify()

notify()

1. הודע **לאחד** החוטים המחכים ב-wait

notifyAll()

1. הודע **לכל** החוטים המחכים ב-wait

wait()

1. שחרר מנעול

2. חכה ל-notify

3. נטול שוב את המנעול

(חכה אם צריך)

המנעול אינו משתחרר בקריאה ל-notify()/notifyAll()

Object-notify() ו-wait() מוגדות ב-Object 

◆ כל עצם יורש את מימושן

■ זוהי שגיאה לקרוא ל-wait() או notify() ללא מנעול

כלומר, מחוץ לבלוק **synchronized** 

wait() ו-notify()

■ כדי לקשר בין notify() ל-wait() שעליו הוא אמור להשפיע:

ה-wait() וה-notify() צריכים להקרא על אותו העצם

שניהם צריכים להקרא אחרי שנלקח אותו המנעול

ה-mobn שעליהם להקרא מחוטים שונים)

```
try
{
    synchronized( object )
    {
        while( ! condition )
            object.wait();
        // condition is true now
    }
}
catch( InterruptedException ie )
{
    .....
}
```

```
synchronized( object )
{
    // make condition true
    object.notify();
}
```

דוגמה : מימוש חיץ עם סינכרוניזציה

```
1. public class SyncBuffer implements Buffer
2. {
3.     private int buffer = -1;
4.     private boolean occupied = false;
5.
6.     public synchronized void set( int value )
7.         throws InterruptedException
8.     {
9.         while( occupied ) {
10.             wait();
11.         }
12.         buffer = value;
13.         occupied = true;
14.         notifyAll();
15.     }
16.
```

```
17.     public synchronized int get()
18.         throws InterruptedException
19.     {
20.         while( !occupied ) {
21.             wait();
22.         }
23.         occupied = false;
24.         notifyAll();
25.         return buffer;
26.     }
27. } /* end class */
```

1. מה המטרה של לולאות ה-while? למה לא מספיק רק בדיקה חד-פעמית ע"י if?
2. אולי ניתן לפתור את הבעיה ע"י שימוש ב-notify במקום ב-notifyAll, ולהימנע מלולאת ה-while?
- מסקנה: לא ניתן להימנע מהלולאה!
3. האם חייבים להשתמש ב-notifyAll?

מהלך תקין

```
1. public class SyncBuffer implements
   Buffer
2. {
3.     private int buffer = -1;
4.     private boolean occupied = false;
5.
6.     public synchronized void set( int
   value )
7.     throws InterruptedException
8.     {
9.         while( occupied ) {
10.            wait();
11.        }
12.        buffer = value;
13.        occupied = true;
14.        notifyAll();
15.    }
16. }
```

```
17. public synchronized int get()
18.     throws InterruptedException
19. {
20.     while (!occupied ) {
21.         wait();
22.     }
23.     occupied = false;
24.     notifyAll();
25.     return buffer;
26. }
27. } /* end class */
```

1. Consumer C2 checks while (!occupied), waits.
2. Consumer C1 checks while (!occupied), waits.
3. Producer P sets buffer = 1, sets occupied = true, calls notifyAll() and releases the lock.
4. Both Consumer C1 and Consumer C2 are awakened by notifyAll().
5. Consumer C2 gets the lock, retrieves buffer value, sets occupied = false, calls notifyAll().
6. Consumer C1 checks while (!occupied), waits again as occupied is now false.

מהלך תקין מפורט

Initial State:

[Buffer: -1] [Occupied: False]

[Producer P (Set(1))] [Consumer C2 (Get())] [Consumer C1 (Get())]

Execution:

1. Consumer C2 arrives, acquires the lock, checks `while (!occupied)`, and waits.

[Buffer: -1] [Occupied: False] Consumer C2 -> acquires lock -> while (!occupied) -> releases lock -> wait()

2. Consumer C1 arrives, acquires the lock, checks `while (!occupied)`, and waits.

Consumer C1 -> acquires lock -> while (!occupied) -> releases lock -> wait()

3. Producer P acquires the lock, checks `while (occupied)`, sets buffer = 1.

Producer P -> acquires lock -> while (occupied) -> set buffer = 1, occupied = true -> notifyAll() -> releases lock

[Buffer: 1] [Occupied: True]

4. Consumer C2 and Consumer C1 are both awakened by `notifyAll()` and attempt to retrieve the lock.

Consumer C2 -> awakened -> acquires lock -> while (!occupied) -> get buffer = 1, occupied = false -> notifyAll() -> releases lock

[Buffer: 1] [Occupied: False]

5. Consumer C1 acquires the lock, re-checks `while (!occupied)`, and waits.

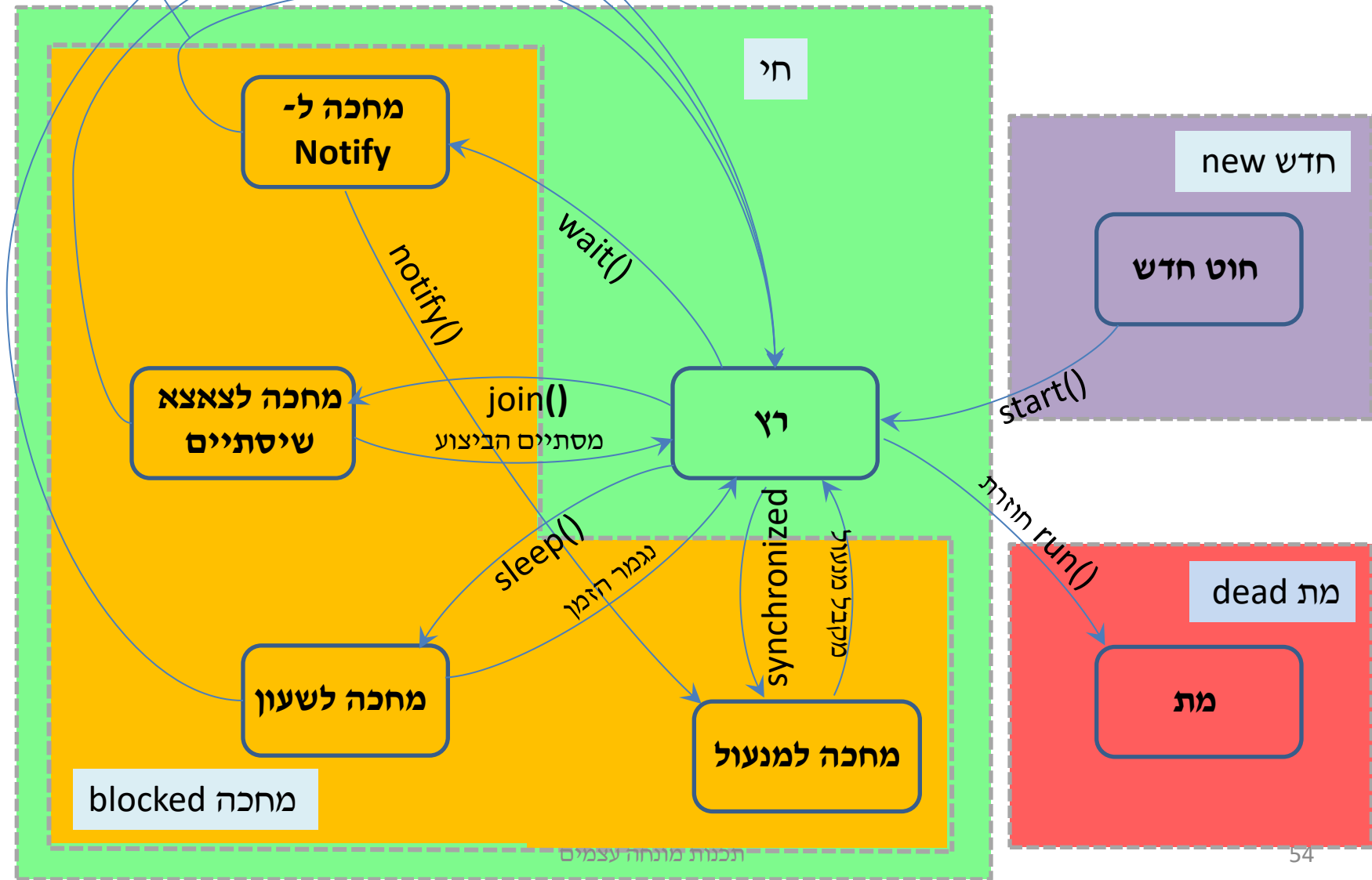
Consumer C1 -> acquires lock -> while (!occupied) -> releases lock -> wait()

[Buffer: 1] [Occupied: False]

```
1. public class SyncBuffer implements Buffer
2. {
3.     private int buffer = -1;
4.     private boolean occupied = false;
5.
6.     public synchronized void set( int value )
7.         throws InterruptedException
8.     {
9.         while( occupied ) {
10.            wait();
11.        }
12.        buffer = value;
13.        occupied = true;
14.        notifyAll();
15.    }
16.
17.     public synchronized int get()
18.         throws InterruptedException
19.     {
20.         while (!occupied ) {
21.            wait();
22.        }
23.        occupied = false;
24.        notifyAll();
25.        return buffer;
26.    }
27. } /* end class */
```

דיאגרמת המצבים

interrupt()



אולי החלפת while ב if?

```
1. public class SyncBuffer implements Buffer
2. {
3.     private int buffer = -1;
4.     private boolean occupied = false;
5.
6.     public synchronized void set( int value )
7.         throws InterruptedException
8.     {
9.         if( occupied ) {
10.             wait();
11.         }
12.         buffer = value;
13.         occupied = true;
14.         notifyAll();
15.     }
16.
```

```
17. public synchronized int get()
18.     throws InterruptedException
19. {
20.     if( !occupied ) {
21.         wait();
22.     }
23.     occupied = false;
24.     notifyAll();
25.     return buffer;
26. }
27. } /* end class */
```

תסריט בעיתי עם if

1. Consumer C1 checks if (!occupied), waits.
2. Consumer C2 checks if (!occupied), waits.
3. Producer P sets buffer = 1, sets occupied = true, calls notifyAll().
4. Both Consumer C1 and Consumer C2 are awakened by notifyAll().
5. Consumer C1 acquires the lock first, retrieves buffer value, sets occupied = false, calls notifyAll().
6. Consumer C2, now awakened, continues execution without checking as occupied is now false and reads buffer again

תסריט בעיתי מפורט

Initial State:

[Buffer: -1] [Occupied: False]

[Consumer C1 (Get())] [Consumer C2 (Get())] [Producer P (Set(1))]

Execution:

1. Consumer C1 acquires the lock, checks `if (!occupied)`, and waits.

[Buffer: -1] [Occupied: False]

Consumer C1 -> acquires lock -> if (!occupied) -> releases lock -> wait()

2. Consumer C2 acquires the lock, checks `if (!occupied)`, and waits.

[Buffer: -1] [Occupied: False]

Consumer C2 -> acquires lock -> if (!occupied) -> releases lock -> wait()

3. Producer P acquires the lock, checks `if (occupied)`, sets buffer = 1.

[Producer P] -> acquires lock -> if (occupied) -> set buffer = 1, occupied = true -> notifyAll() -> releases lock

[Buffer: 1] [Occupied: True]

4. Consumer C1 and Consumer C2 are both awakened by `notifyAll()` and attempt to retrieve the lock.

Consumer C1 -> awakened, tries to acquire lock

Consumer C2 -> awakened, tries to acquire lock

5. Consumer C1 acquires the lock first and accesses the buffer.

Consumer C1 -> acquires lock -> get buffer = 1, occupied = false -> notifyAll() -> releases lock

[Buffer: 1] [Occupied: False]

6. Consumer C2 acquires the lock after Consumer C1 and accesses the buffer.

Consumer C2 -> acquires lock -> get buffer = 1, occupied = false -> notifyAll() -> releases lock

[Buffer: 1] [Occupied: False]

```
1. public class SyncBuffer implements Buffer
2. {
3.     private int buffer = -1;
4.     private boolean occupied = false;
5.
6.     public synchronized void set( int value )
7.         throws InterruptedException
8.     {
9.         if( occupied ) {
10.            wait();
11.        }
12.        buffer = value;
13.        occupied = true;
14.        notifyAll();
15.    }
16.
17.     public synchronized int get()
18.         throws InterruptedException
19.     {
20.         if( !occupied ) {
21.            wait();
22.        }
23.        occupied = false;
24.        notifyAll();
25.        return buffer;
26.    }
27. } /* end class */
```

קריאה
פעמיים של 1

אולי notify במקום notifyAll?

```
1. public class SyncBuffer implements Buffer
2. {
3.     private int buffer = -1;
4.     private boolean occupied = false;
5.
6.     public synchronized void set( int value )
7.         throws InterruptedException
8.     {
9.         if( occupied ) {
10.            wait();
11.        }
12.        buffer = value;
13.        occupied = true;
14.        notify();
15.    }
16.
17.     public synchronized int get()
18.         throws InterruptedException
19.     {
20.         if( !occupied ) {
21.            wait();
22.        }
23.        occupied = false;
24.        notify();
25.        return buffer;
26.    }
27. } /* end class */
```

תסריט בעיתי 2 עם notify במקום notifyAll

1. Consumer C1 checks if (!occupied), waits.
2. Producer P acquires the lock, starts setting buffer = 1.
3. Consumer C2 arrives and is blocked at the entrance to the method.
4. Producer P sets buffer = 1, sets occupied = true, calls notify(), and releases the lock.
5. Consumer C2 acquires the lock, retrieves buffer value, calls notify(), and releases the lock.
6. Consumer C1 is awakened by notify(), acquires the lock, retrieves buffer value.

תסריט בעיתי 2 מפורט

Initial State:

[Buffer: -1] [Occupied: False]

[Consumer C1 (Get())] [Consumer C2 (Get())] [Producer P (Set(1))]

Execution:

1. Consumer C1 checks if (!occupied), waits.

[Buffer: -1] [Occupied: False]

Consumer C1 -> acquires lock -> if (!occupied) -> releases lock -> wait()

2. Producer P acquires the lock, starts setting buffer = 1.

Producer P -> acquires lock -> set buffer = 1

3. Consumer C2 arrives and is blocked at the entrance to the method.

Consumer C2 -> blocked at method entrance (waiting for lock which is held by A)

4. Producer P sets occupied = true, calls notify(), and releases the lock.

Producer P -> sets occupied = true -> notify() -> releases lock

[Buffer: 1] [Occupied: True]

5. Consumer C2 acquires the lock, checks if (!occupied), retrieves buffer value, calls notify(), and releases the lock.

Consumer C2 -> acquires lock -> if (!occupied) -> get buffer = 1 -> notify() -> releases lock

[Buffer: 1] [Occupied: False]

6. Consumer C1 is awakened by notify(), acquires the lock, and retrieves buffer value without checking if (!occupied).

Consumer C1 -> awakened -> acquires lock -> get buffer = 1 -> releases lock

[Buffer: 1] [Occupied: False]

```
1. public class SyncBuffer implements Buffer
2. {
3.     private int buffer = -1;
4.     private boolean occupied = false;
5.
6.     public synchronized void set( int value )
7.         throws InterruptedException
8.     {
9.         if( occupied ) {
10.            wait();
11.        }
12.        buffer = value;
13.        occupied = true;
14.        notify();
15.    }
16.
17.     public synchronized int get()
18.         throws InterruptedException
19.     {
20.         if (!occupied) {
21.             wait();
22.         }
23.         occupied = false;
24.         notify();
25.         return buffer;
26.     }
27. } /* end class */
```

קריאה
פעמיים של 1

סיכום

Condition check	Notification	Correct?	Risk
if	notify()	✗ No	double read
if	notifyAll()	✗ No	logic bug
while	notify()	⚠ Sometimes	deadlock / starvation
while	notifyAll()	✓ Yes	safe

הפרדה של wait מ-lock

■ השתמשנו בסינכרוניזציה לשתי מטרות:

■ מניעת גישה בו זמנית משני חוטים למשאב משותף

◆ ע"י נעילה של אזורים קריטיים באמצעות **synchronized**

◆ (תוך הקפדה על שימוש ב**אותו המנעול** בכל המקרים)

■ כפיה של ביצוע אזורים קריטיים בסדר מסויים

◆ ע"י עכוב ביצוע בטרם עת ע"י wait()

◆ ושחרור העכוב ע"י notify()

■ יש אפשרות לבצע פעולות אלו בנפרד, על ידי מחלקות יעודיות

■ Lock – מנשק של מנעול

◆ lock() ו-unlock()

■ Condition – מנשק הכולל מנגנון עיכוב ושחרור

◆ await() ו-signal()

Lock – מנשק המתאר מנעול

■ מתודות:

lock() – החוט הקורא מבקש לרכוש את המנעול

◆ המתודה חוזרת רק אחרי שהמנעול נרכש

◆ אם המנעול תפוס, מחכה עד שישתחרר

tryLock() – החוט הקורא מבקש מנעול רק אם הוא פנוי

◆ המתודה חוזרת מיד

◆ אם המנעול פנוי, הוא נרכש והמתודה מחזירה **true**

◆ אם המנעול תפוס, המתודה מחזירה **false**

unlock() – משחררת את המנעול

■ מחלקות מממשות:

ReentrantLock – מנעול מהסוג שעליו דיברנו עד כה

ReentrantReadWriteLock – מנעול קריאה/כתיבה

Condition – מנשק המתאר תנאי

■ מופעים נוצרים ע"י מתודת Factory במחלקה המממשת את Lock :

```
Condition cond = lock.newCondition();
```

למה לא נוצרים באופן רגיל?

■ כדי לחכות לקיום תנאי כלשהו φ :

```
while( !  $\varphi$  )
```

```
    cond.await();
```

■ כדי לשחרר חוט המחכה לקיום תנאי :

```
cond.signal();
```

■ כדי לשחרר כל החוטים המחכים :

```
cond.signalAll();
```

הפרדה של wait מ-lock

```
class LockedBuffer {  
    final Lock lock = new ReentrantLock();  
    final Condition cond = lock.newCondition();  
    int buffer = -1;  
    boolean occupied = false ;
```

אין synchronized

```
    public void set( int value )  
        throws InterruptedException {  
        lock.lock();  
        try {  
            while( occupied )  
                cond.await();  
            buffer = value;  
            occupied = true;  
            cond.signalAll();  
        }  
        finally {  
            lock.unlock();  
        }  
    }  
}
```

```
    public int get()  
        throws InterruptedException {  
        int value;  
        lock.lock();  
        try {  
            while( ! occupied )  
                cond.await();  
            value = buffer;  
            occupied = false;  
            cond.signalAll();  
            return( value );  
        }  
        finally {  
            lock.unlock();  
        }  
    }  
}
```

הפרדה של wait מ-lock

```
class LockedBuffer {  
    final Lock lock = new ReentrantLock();  
    final Condition full = lock.newCondition();  
    final Condition empty = lock.newCondition();
```

```
    int buffer = -1;  
    boolean occupied = false;
```

```
    public void set( int value )  
        throws InterruptedException {  
        lock.lock();  
        try {  
            while( occupied )  
                empty.await();  
            buffer = value;  
            occupied = true;  
            full.signal();  
        }  
        finally {  
            lock.unlock();  
        }  
    }  
}
```

אין synchronized

```
    public int get()  
        throws InterruptedException {  
        int value;  
        lock.lock();  
        try {  
            while( ! occupied )  
                full.await();  
            value = buffer;  
            occupied = false;  
            empty.signal();  
            return( value );  
        }  
        finally {  
            lock.unlock();  
        }  
    }  
}
```

מחיה

■ נתן למנוע התנגשות באופן לוקלי

■ בעיות מחיה הן גלובליות:

■ הרעבה (Starving) – חוט צריך לרוץ אך תמיד יש זריזים ממנו

■ נעילה – חוט נשאר במצב wait ואף אחד לא מעיר אותו

■ קפאון (deadlock) – שני חוטים אוחזים במנעול שהאחר צריך

◆ חוט אחד אוחז במנעול אחד, חוט שני אוחז מנעול שני,

◆ וכל אחד ממתין לשחרור המנעול שנמצא בידי החוט האחר.

■ בעיות יותר קשות לזיהוי ופתרון

קיפאון (deadlock) - דוגמה

```
public class ThreadDemo extends Thread {
    private Integer lock1;
    private Integer lock2;
    private int id;
    public ThreadDemo(int id, Integer lock1, Integer lock2) {
        this.id=id;  this.lock1=lock1;  this.lock2=lock2;
    }
    public void run() {
        synchronized (lock1) {
            System.out.println("Thread" +id+ ": Holding lock " + lock1);
            try { Thread.sleep(10); }
            catch (InterruptedException e) {}
            System.out.println ("Thread" +id+ ": Waiting for lock " + lock2);
            synchronized (lock2) {
                System.out.println ("Thread" +id+ ": Holding lock " + lock1 +" & " + lock2);
            } //lock2
        } //lock1
    } //run()
} //class
```

```
public class DeadLock {
    public static void main(String args[]) {
        Integer l1 = new Integer (1);
        Integer l2 = new Integer (2);
        ThreadDemo t1 = new
            ThreadDemo(1, l1, l2);
        ThreadDemo t2 = new
            ThreadDemo(2, l2, l1);
        t1.start(); t2.start();
    }
}
```

סיכום

- חוטים מאפשרים ריצה של כמה זרמי פקודות במקביל
 - מתאים לישומים מגיבים
 - מנצל יכולות של החומרה
- חוט קל יותר מתהליך, וכמה מהם מוכלים בתהליך אחד
- מוגדרים ע"י הורשה מ-Thread או מימוש Runnable
- חוט הוא באחד מהמצבים: חדש, רץ, מחכה או מת.
- אזור קריטי חייב להתבצע בשלמותו בחוט אחד בבת אחת
- סינכרוניזציה: **synchronized**, wait(), notify()
 - אפשר גם עם Lock ו-Condition
- מחיה: הרעבה, נעילה וקפאון

שאלות

■ מה ההבדל בין sleep(...) ל-wait(...)?

■ כיצד חוזרים למצב ריצה?

■ מה בקשר למנעול?

■ האם המתודה יכולה או/ו צריכה להיקרא בתוך בלוק מסונכרן?

■ באיזו מחלקה מוגדרת המתודה?

■ למה wait(), notify(), notifyAll() צריכות להיקרא מתוך בלוק מסונכרן?

■ מהי פעולה אטומית? כיצד ניתן להגדיר זאת?

שאלון